DIGITAL ELECTRONICS

ET 8301

Friday, November 8, 2024

Mr. Cuthbert J. Karawa

Lecture 02

NUMBER SYSTEMS

College of Information and Communication Technology (CoICT)

Department Of Electronics And Telecommunications Engineering.

Introduction

- **Digital systems** are used to process data and to perform calculations in most instrumentation, monitoring and communication devices.
- As **physical quantities** and **signals** can only take **discrete values** in a digital system, the interpretation of real-world information requires the use of interface circuits such as **data converters**.
- In general, numbers may be represented in different numeration systems. The **decimal system** is commonly used in routine transactions while the **binary system** is the basis for digital electronics.

Introduction

- Every number (or numeration) system is defined by a **base** (or ***radix***), which is a collection of distinct symbols.
- The representation of a number in a numeration system may be considered as a change in base.
- In a positional number system, a value of a number depends on ***the place occupied by each of its digits*** in the representation

Decimal numbers

- The decimal number system uses the following ***10 numbers or symbols*** : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. The radix is thus **10.**
- The place values of different digits in mixed decimal number, starting from the decimal point, are 10^0 ,10^1,10^2 and so on (for the integer part) and 10^{-1},10^{-2},10^{-3} and so on (for the fractional part)
- For Example , Decompose the numbers 734 and 12345 into powers of 10.

  The decomposition of the number 734 takes the form:

  $734 = (7 \times 10^2) + (3 \times 10^1) + (4 \times 10^0)$

  $= 734_{10}$

Decimal numbers

For the number 12345, we have:

$12345 = (1\times10^4) + (2\times10^3) + (3\times10^2) + (4\times10^1) + (5\times10^0)$

$=12345_{10}$

- Depending on its position, each number is ***multiplied*** by the appropriate power of ***10***. The right-most digit represents the unit digit.
- For Example, for decimal number 2785.618, the integer part (i.e 2785) can be expressed as

$2785=5\times10^0 + 8\times10^1 + 7\times10^2 + 2\times10^3$

$=5+80+700+2000=27850$

- And fractional part can be expressed as

$618=6\times10^{-1} + 1\times10^{-2} + 8\times10^{-3}$

$=0.6+0.01+0.008=0.618$

Binary numbers

- Binary number system is based on ***two-level logic(base 2)***, conventionally noted as **0** (low level) and **1** (high level). It is a system with a radix of two.
- All larger binary number are represented in terms of ‘0’ and ‘1’.
- Counting in binary, is the same as decimal, the only exception is that decimal uses ten digits and binary only has two digits (0 and 1).
- If we were counting using ***four bits***, then the sequence would look like: 0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001,1010, 1011, 1100, 1101, 1110, and 1111.
- With two symbols for each bit, we have 2^n possible combinations of symbols where n is the number of bits.

Binary numbers

A bit	• is a single place or position in a binary number
Nibble	• A four bit binary number
Byte	• An eight bit (2 nibble) binary number
Word	• Is a string of bits whose size (the word length) may be equal to one byte, two bytes, four bytes and more.

Binary numbers

- The rightmost bit, the one that represents the ones place, is called the **Least Significant Bit** or **LSB**.

- The leftmost bit, the one that represents the highest power of two represented by that number, is called the **Most Significant Bit** or **MSB**

Binary numbers

- For example : Convert the decimal numbers 13 and 125 into binary numbers
  - The decomposition of the number 13 in powers of 2 is written as: $13_{10} = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$
    $= 1101_2$
  - For the number 125, we have:
    $125_{10} = (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (0 \times 21)$
    $+(1 \times 2^0) = 1111101_2$

Binary numbers

- The conversion of a decimal number to a binary number can be carried out as follow

$$
\begin{aligned}
13 \div 2 &= 6 \quad \text{Remainder 1 (LSB)} \\
6 \div 2 &= 3 \quad \text{Remainder 0} \\
3 \div 2 &= 1 \quad \text{Remainder 1} \\
1 \div 2 &= 0 \quad \text{Remainder 1 (MSB)}
\end{aligned}
$$

$$13_{10} = 1101_2$$

$$
\begin{aligned}
125 \div 2 &= 62 \quad \text{Remainder 1 (LSB)} \\
62 \div 2 &= 31 \quad \text{Remainder 0} \\
31 \div 2 &= 15 \quad \text{Remainder 1} \\
15 \div 2 &= 7 \quad \text{Remainder 1} \\
7 \div 2 &= 3 \quad \text{Remainder 1} \\
3 \div 2 &= 1 \quad \text{Remainder 1} \\
1 \div 2 &= 0 \quad \text{Remainder 1 (MSB)}
\end{aligned}
$$

$$125_{10} = 1111101_2$$

Octal numbers

- The octal number system has a base of 8 and therefore has eight distinct digits.
- All higher-order number are expressed as a combination of these on the same pattern as the one followed in the case of the binary and decimal number systems.
- The independent digits are 0,1,2,3,4,5,6, and 7.
- The next 10 numbers that follow ‘7’, for example, would be 10,11, 12, 13. 14, 15, 16, 17, 20 and 21

Octal numbers

- For **example** Convert the decimal numbers 250 and 777 to octal numbers.

- In radix 8 representation, the number 250 takes the form:

$$250_{10} = (3 \times 8^2) + (7 \times 8^1) + (2 \times 8^0)$$
$$= 372_8$$

  - In the case of the number 777, we have:

$$777_{10} = (1 \times 8^3) + (4 \times 8^2) + (1 \times 8^1) + (1 \times 8^0)$$
$$= 1411_8$$

$$250 \div 8 = 31 \quad \text{Remainder 2 (LSD)}$$
$$31 \div 8 = 3 \quad \text{Remainder 7}$$
$$3 \div 8 = 0 \quad \text{Remainder 3 (MSD)}$$

$$250_{10} = 372_8$$

$$777 \div 8 = 97 \quad \text{Remainder 1 (LSD)}$$
$$97 \div 8 = 12 \quad \text{Remainder 1}$$
$$12 \div 8 = 1 \quad \text{Remainder 4}$$
$$1 \div 8 = 0 \quad \text{Remainder 1 (MSD)}$$

$$777_{10} = 1411_8$$

Hexadecimal numbers

- The hexadecimal number system is a **base-16** number system and its **16 basic digits** are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.
- The place values or weights of different digits in a mixed hexadecimal number are 16^0, 16^1, 16^2 and so on (for the integer part) and 16^{-1}, 16^{-2},16^{-3} and so on (for the fractional part)
- For ***example*** Convert the decimal numbers 31 and 2988 into hexadecimal.
  - To obtain the equivalent hexadecimal from the binary representation, each group of four bits is replaced by the corresponding hexadecimal digit. We therefore have:

$$31_{10} = 11111_2 = \underbrace{0001}_{1}\ \underbrace{1111}_{15=F} = 1F_{16}$$

Hexadecimal numbers System

- Similarly,

$$2\,988_{10} = 101110101100_2 = \underbrace{1011}_{11=B}\ \underbrace{1010}_{10=A}\ \underbrace{1100}_{12=C} = BAC_{16}$$

- It is generally more convenient to represent the value of an octet using two hexadecimal digits as it is more compact

Conversion Between Number Systems

- **Decimal-to-binary Conversion**.
  - Conversion of decimal to a binary number can be done by dividing the decimal number by 2 repeatedly, until the quotient of zero is obtained.
  - This method is called the '***double-dabble***' method.
  - The remainders are ***noted down*** for each of the division steps.

Conversion Between Number Systems

- **Decimal-to-binary Conversion.**
  - **Example 1**.

    Convert 26_{10} into a binary number

Division	Reminder	
$2\lfloor\underline{26}$	0	
$2\lfloor\underline{13}$	1	
$2\lfloor\underline{6}$	0	
$2\lfloor\underline{3}$	1	
$2\lfloor\underline{1}$	1	↑
0		

$26_{10} = (11010)_2$

Conversion Between Number Systems

- **Decimal-to-octal Conversion**.
  - Conversion of decimal to a ***Octal number*** can be done by dividing the decimal number by 8 repeatedly, until the quotient of zero is obtained.
  - This method of repeated division by 8 is called '***octal-dabble***'.
  - The remainders are ***noted down*** for each of the division steps.

Conversion Between Number Systems

- **Decimal-to-octal Conversion**.
  - ***Example 2***.

    Convert 426_{10} into an octal number.

Division	Reminder	
$8\overline{)426}$	2	
$8\overline{)53}$	5	
$8\overline{)6}$	6	↑
0		

$$426_8 = 652_{10}$$

Conversion Between Number Systems

- **Decimal-to-hexadecimal Conversion**.
  - Conversion of decimal to a Hexadecimal number can be done by dividing the decimal number by 16 repeatedly, until the quotient of zero is obtained.
  - This method of repeated division by ***16*** is called '***hex-dabble***'.
  - The remainders are ***noted down*** for each of the division steps.

Conversion Between Number Systems

- **Decimal-to-hexadecimal Conversion**.
  - ***Example 3***.

    Convert 348_{10} into a hexadecimal number.

Division	Reminder
$16\overline{)348}$	12=C
$16\overline{)21}$	5
$16\overline{)1}$	1 ↑
0	

$$348_{10} = 15\boldsymbol{C}_{16}$$

Conversion Between Number Systems

➢ **Decimal Equivalent**.

- Reverse method, i.e., the method of conversion of binary, octal, or hexadecimal numbers to decimal numbers.
- The decimal equivalent of a given number in another number system is given by ***the sum of all the digits multiplied by their respective place values***. The integer and fractional parts of the given number should be ***treated separately***.
- Binary-to-decimal, octal-to-decimal and hexadecimal-to-decimal conversions are illustrated below with the help of examples.

Conversion Between Number Systems

- **Binary-to-decimal Conversion**.
  - ***Example 4***.

    Convert 1001.0101_2 into a decimal number.

    Integer part

    $$1001 = 1\times 2^3 + 0\times 2^2 + 0\times 2^1 + 1\times 2^0$$
    $$= 8 + 0 + 0 + 1 = 9$$

    The fractional part

    $$.0101 = 0\times 2^{-1} + 1\times 2^{-2} + 0\times 2^{-3} + 1\times 2^{-4}$$
    $$= 0 + 0.25 + 0 + 0.0625 = 0.3125$$

    $$(1001.0101)_2 = 9.3125_{10}$$

Conversion Between Number Systems

- **Octal-to-decimal Conversion**.
  - ***Example 5***.

    Convert $(137.21)_8$ into a decimal number

The integer part

$$137 = 7\times8^0 + 3\times8^1 + 1\times8^2 = 7 + 24 + 64 = 95$$

The fractional part

$$.21 = 2\times8^{-1} + 1\times8^{-2} = 0.265$$

$$(137.21)_8 = (95.265)_{10}$$

Conversion Between Number Systems

- **Hexadecimal-to-decimal Conversion**.
  - ***Example 6***.

    Convert $(1E0.2A)_{16}$ into a decimal number.

    The integer part

    $$1E0 = 0\times16^{0} + 14\times16^{1} + 1\times16^{2}$$
    $$= 0 + 224 + 256 = 480$$

    The fractional part

    $$2A = 2\times16^{-1} + 10\times16^{-2} = 0.164$$

    $$(1E0.2A)_{16} = (480.164)_{10}$$

Conversion Between Number Systems

- **Binary–Octal Conversions**.
  - The maximum digit in an octal number system is 7, which can be represented as 111_2 in a binary system.
  - Therefore, starting from the **LSB**, group three digits at a time and replace them by the decimal equivalent of those groups to get the ***final octal number***.

Conversion Between Number Systems

- **Binary–Octal Conversions**.
  - ***Example 5***.

    Convert $(1110100.0100111)_2$ into an equivalent octal number.

$$(1110100.0100111)_2$$

$$001 \vdots 110 \vdots 100.010 \vdots 011 \vdots 100$$

$$1 \quad 6 \quad 4 \; . \; 2 \quad 3 \quad 4$$

$$(1110100.0100111)_2 = (164.234)_8$$

Conversion Between Number Systems

- **Octal–Binary Conversions**.
  - The conversion from octal number to binary equivalent, is done by convert octal digit into a 3-bit-equivalent binary number and combining all those digits to get the final binary equivalent.

Conversion Between Number Systems

- **Octal–Binary Conversions**.
  - ***Example 5***.

    Convert 473.21_8 into an equivalent binary number.

    The octal number given is 4 7 3 . 2 1

    3-bit binary equivalent 100 111 011 . 010 001

    473.21_8=100111011.010001

Conversion Between Number Systems

- **Binary–Hex Conversions**.
  - The maximum digit in a hexadecimal system is 15, which can be represented by1111_2 in a binary system.
  - Hence, starting from the LSB, group four digits at a time and replace them with the hexadecimal equivalent of those groups to get the final hexadecimal number.

Conversion Between Number Systems

- **Binary–Hex Conversions**.
  - ***Example 5***.

    Convert 111011.0112 into an equivalent hexadecimal number.

    The binary number given is 111011.011

    Grouping 4 bits 0011 1011. 0110

    Hexadecimal equivalent 3 B 6

    $(111011.011)_2 = (3B.6)_{16}$

Conversion Between Number Systems

- **Hex-Binary Conversions**.
  - The hexadecimal number is converted into its binary equivalent, then each hexadecimal digit is converted into a 4- bit-equivalent binary number and by combining all those digits we get the final binary equivalent.

Conversion Between Number Systems

- **Hex-Binary Conversions**.
  - ***Example*** .

    Convert 9E.AF216 into an equivalent binary number.

	9	E	.	A	F	2
4-bit binary equivalent	1001	1110	.	1010	1111	0010

$(9E.AF2)_8=(10011110.101011110010)_2$

Conversion Between Number Systems

- **Conversion from an Octal to Hexadecimal Number and Vice Versa**.
  - To convert an octal number into a hexadecimal number the following steps are to be followed:
    - ✓ First convert the octal number to its binary equivalent
    - ✓ Then form groups of 4 bits, starting from the LSB.
    - ✓ Then write the equivalent hexadecimal number for each group of 4 bits.
  - Similarly, for converting a hexadecimal number into an octal number the following steps are to be followed
    - ✓ First convert the hexadecimal number to its binary equivalent.
    - ✓ Then form groups of 3 bits, starting from the LSB.
    - ✓ Then write the equivalent octal number for each group of 3 bits.

Conversion Between Number Systems

- **Conversion from an Octal to Hexadecimal Number and Vice Versa**.
  - **Example.**

Convert the 4.BF85 hexadecimal numbers into equivalent octal

	4.	B	F	8	5
Binary equivalent is	0100.	1011	1111	1000	0101
	= 0100.1011111110000101				
Forming groups of 3 bits	100.	101	111	111	000	010	100
Octal equivalent	4.	5	7	7	0	2	4

$\therefore \quad (4.BF85)_{16}$ is $(4.577024)_8$

Conversion Between Number Systems

- **Conversion from an Octal to Hexadecimal Number and Vice Versa**.
  - **Example.**

Convert $(36.532)_8$ into an equivalent hexadecimal number

	3	6.	5	3	2
Binary equivalent is	011	110.	101	011	010
	011110.101011010				
Forming groups of 4 bits	0001	1110.	1010	1101	
Hexadecimal equivalent	1	E.	A	D	

$(36.532)_8 = (1E.AD)_{16}$

Quiz

1. Convert 0.7520_{10} to binary.
2. Convert 0.245_{10} to binary.
3. Convert 341.32_4 to radix 7

Complements

- **Binary Number System**.
  - The 1's complement of a binary number is obtained by complementing all its bits, i.e. by replacing 0's with 1's and 1's with 0's.
  - For example, the 1's complement of $(10010110)_2$ is $(01101001)_2$.
  - The 2's complement of a binary number is obtained by adding '1' to its 1's complement.
  - The 2's complement of $(10010110)_2$ is $(01101010)_2$.

Complements

- **Decimal Number System**.
  - In the decimal number system, we have the 9's and 10's complements.
  - The 9's complement of a given decimal number is obtained by subtracting each digit from 9.
  - For example, the 9's complement of $(2496)_{10}$ would be $(7503)_{10}$.
  - The 10's complement is obtained by adding '1' to the 9's complement. The 10's complement of $(2496)_{10}$ is $(7504)_{10}$.

Complements

- **Octal Number System**.
  - In the octal number system, we have the 7's and 8's complements.
  - The 7's complement of a given octal number is obtained by subtracting each octal digit from 7.
  - For example, the 7's complement of $(562)_8$ would be $(215)_8$. The 8's complement is obtained by adding '1' to the 7's complement.
  - The 8's complement of $(562)_8$ would be $(216)_8$.

Complements

- **Hexadecimal Number System**.
  - The 15's and 16's complements are defined with respect to the hexadecimal number system.
  - The 15's complement is obtained by subtracting each hex digit from 15.
  - For example, the 15's complement of $(3BF)_{16}$ would be $(C40)_{16}$.
  - The 16's complement is obtained by adding '1' to the 15's complement.
  - The 16's complement of $(2AE)_{16}$ would be $(D52)_{16}$.

Arithmetic operations on binary numbers

- Arithmetic operations on ***binary numbers*** may be executed in the same way as for decimal numbers.
- The ***addition*** is the most executed arithmetic operation in digital systems.
- The ***subtraction*** operation is essentially a variant of the addition operation,
- While ***multiplication*** and ***division*** operations may be carried out by combining logical functions (AND, OR, shift, etc.) and addition.

Arithmetic operations on binary numbers

- **Addition**.
  - In binary representation, we begin by adding bits of lower weight, and the carry that may be obtained when the sum of bits of the same weight exceeds the highest value that can be represented with one bit, that is 1, is transferred, each time, to the next MSB.
  - In binary representation, addition is carried out according to the following rules:

$$
\begin{aligned}
0 + 0 &= 0 \\
0 + 1 = 1 + 0 &= 1 \\
1 + 1 &= 0 \quad \text{Carry 1} \\
1 + 1 + 1 &= 1 \quad \text{Carry 1}
\end{aligned}
$$

Arithmetic operations on binary numbers

- **Addition**.
  - Example Add the numbers 1010 and 1011.
    Carrying out the addition operation in binary and decimal, we have:

$$\begin{array}{r} 1011 \\ +\ 0011 \\ \hline 1110 \end{array} \qquad \begin{array}{r} 11 \\ +\ 3 \\ \hline 14 \end{array}$$

    The sum is obtained by adding the numbers, each of which is called the *addend*.
  - In practice, more than two numbers can be added in a digital system by initially determining the sum of the first two numbers, then adding this sum to the third number and so on.

Arithmetic operations on binary numbers

- **Subtraction**.
  - In binary representation, the execution of a subtraction operation takes place ***from the LSBs to the MSBs*** with the assumption that the number to be subtracted (or the subtrahend) is the smaller of the two operands.
  - The difference is the result obtained upon subtracting the subtrahend from the minuend.
  - Before subtracting a number (bit at the logic level 1) from another number of lower value (bit at the logic level 0), we add the value of the radix (that is 2) to the latter and a borrow of 1 is then carried over to the next highest bit to be subtracted. The rules governing binary subtraction are:

Arithmetic operations on binary numbers

- **Subtraction**.
  - The rules governing binary subtraction are:

$$0 - 0 = 0$$
$$0 - 1 = 1 \quad \text{Borrow } 1$$
$$1 - 0 = 1$$
$$1 - 1 = 0$$

Arithmetic operations on binary numbers

- **Subtraction**.
  - Example: Subtract the number 101 from the number 1010.

    The subtraction may be carried out in binary representation and in decimal representation as follows:

1010	10	Minuend
− 0101	− 5	Subtrahend
0101	5	Difference

```
          1000
        − 1001
end carry → 1 1111
```

Arithmetic operations on binary numbers

- **Subtraction**.
  - The above method of subtraction is know as ***the direct method***.
  - When the minuend is smaller than the subtrahend the result of subtraction is negative and in the direct method the result obtained is in 2's complement form.
  - So to get back the actual result we have to perform the 2's complement again on the result thus obtained.

Arithmetic operations on binary numbers

➢ Subtraction Using 1's Complement .

- Binary subtraction can be performed by adding the 1's complement of the subtrahend to the minuend.
- If a carry is generated, remove the carry, add it to the result.
- To get the true result we have to make the 1's complement of the result obtained and put a negative sign before the result.
- ***Example***: Subtract $(1001)_2$ from $(1101)_2$ and $(1100)_2$ from $(1001)_2$ using the 1's complement method.

```
                1101
1's complement → 0110
        Carry → 10011
  Add Carry →       1
                0100
```

```
                    1001
1's Complement →    0011
                    1100
1's Complement      0011
True result = – 0011 or 10011
```

Arithmetic operations on binary numbers

- **Subtraction Using 2's Complement**.
  - Binary subtraction can be performed by adding the 2's complement of the subtrahend to the minuend.
  - If a carry is generated, discard the carry.
  - Now if the subtrahend is larger than the minuend, then no carry is generated.
  - The answer obtained is in 2's complement and is negative.
  - To get a true answer take the 2's complement of the number and change the sign.

Arithmetic operations on binary numbers

- **Subtraction Using 2's Complement**.
  - Example: Subtract $(1010)_2$ from $(1001)_2$ using the 2's complement method.

$$
\begin{array}{lll}
 & & 1001 \\
\text{2's complement} & \rightarrow & \underline{0110} \\
 & & 1111 \\
\text{2's complement} & \rightarrow & 0001 \\
\text{Answer} & & -0001
\end{array}
$$

Arithmetic operations on binary numbers

- **Multiplication**.
  - Multiplication is carried out by forming a partial product for each ***bit of the multiplier*** and then adding all the partial products to generate the result.
  - It must be **noted** that each partial product is shifted one position to the left with respect to the preceding one and the product of two n-bit numbers may possess up to 2n bits.
  - The multiplication table in binary representation can be summarized as follows:

$$0 \times 0 = 0$$
$$0 \times 1 = 0$$
$$1 \times 0 = 0$$
$$1 \times 1 = 1$$

Arithmetic operations on binary numbers

- **Multiplication**.
  - Example Multiply the number 1101 by 1001.
    Executing multiplication in binary representation translates to:

	1101	Multiplicand
×	1001	Multiplier
	1101	First partial product
	0000	Second partial product
	0000	Third partial product
+	1101	Fourth partial product
	1110101	Product

Arithmetic operations on binary numbers

- **Division**.
  - Division of a binary number (the dividend) by another (the divisor) is carried out by repeatedly deducting the divisor from the dividend until you obtain a difference that is either equal to zero or inferior to the divisor and that represents the remainder.
  - The quotient corresponds to the number of times the divisor is contained in the dividend.
  - When the dividend is a ***2n-bit*** number and the divisor is an n-bit number, the quotient may be represented as an n-bit number.
  - Division is executed by comparing the ***n bits*** of the ***divisor*** with the ***n LSBs*** of the dividend.

Arithmetic operations on binary numbers

- **Division**.
  - If the divisor is ***greater than*** the dividend, no subtraction is performed, the corresponding quotient bit is set to ***0***, and the divisor is then compared to the ***n + 1*** **LSBs** of the dividend.
  - If, on the other hand, the divisor is ***less than or equal*** to the considered dividend bits, a subtraction is carried out and the corresponding quotient bit is set to ***1***.
  - The comparison process of the divisor continues with the number obtained by lowering the next MSB of the dividend to the right of the previously obtained difference.

Arithmetic operations on binary numbers

- **Division**.
  - Example Divide the following binary numbers: (a) 11001 and 101

```
        101
   ------------
101 ) 11001
      101
      -----
       0010 1
       0010 1
       -----
       0000
```

Weighted Binary Codes

➢ BCD Code or 8421 Code

- Binary coded decimal (BCD) is a weighted code that is commonly used in digital systems when it is necessary to show decimal numbers such as in clock Displays
- Four bits are required to code each decimal number

Decimal	Binary	BCD	
	0000	0000	0000
1	0001		0001
2	0010		0010
3	0011		0011
4	0100		0100
5	0101		0101
6	0110		0110
7	0111		0111
8	1000		1000
9	1001		1001
10	1010	0001	0000
11	1011	0001	0001
12	1100	0001	0010
13	1101	0001	0011
14	1110	0001	0100
15	1111	0001	0101

Weighted Binary Codes

BCD-to-Binary Conversion

- A given BCD number can be converted into an equivalent binary number by first writing its decimal equivalent and then converting it into its binary equivalent

Binary-to-BCD Conversion

- A given binary number can be converted into an equivalent BCD number by first determining its decimal equivalent and then writing the corresponding BCD quivalent

Nonweighted Codes

➢ Excess-3 Code

- The excess-3 code for a given decimal number is determined by adding ‘3’ to each decimal digit in the given number and then replacing each digit by its four-bit binary equivalent.

Decimal number	Excess-3 code	Decimal number	Excess-3 code
0	0011	5	1000
1	0100	6	1001
2	0101	7	1010
3	0110	8	1011
4	0111	9	1100

Nonweighted Codes

➢ Gray Code

- Gray code is unweighted and is not arithmetic code.
- In Gray code a number changes by only one bit as it proceeds from one number to the next.
- This code finds extensive use for shaft encoders, in some types of analog to digital converters.
- We convert binary number to gray code to reduce the switching operation

Decimal	Binary	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Nonweighted Codes

- **Binary to Gray Code Conversion**

  Step 1 : Record MSB as it is

  Step 2 : Add the MSB to the next bit, record the sum and neglect the carry.

  Step 3 : Repeat the process.

HOME WORK

- Gray Code to Binary Conversion
- Study on ASCII codes.
- Pass through different books and do one example from different source and Document it.

Next Lecture

BASIC LOGIC GATES